Thomas Nguyen
11888002
Professor Zhe Dang
Due 4/27/26

---

1. First read RSA lecutre notes. Now, this is what you know: my public key: (e = 49; n = 10539750919), tell me what is the private key? (It is ok if you download some tools or code from the Internet to break the code)

Factor n:

$$n = p \times q = 43{,}481 \times 242{,}399$$

Compute $\phi(n)$:

$$\phi(n) = (p-1)(q-1) = 43{,}480 \times 242{,}398 = 10{,}539{,}465{,}040$$

Find d using the Extended Euclidean Algorithm, such that:

$$e \cdot d \equiv 1 \pmod{\phi(n)}$$

$$49 \cdot d \equiv 1 \pmod{10{,}539{,}465{,}040}$$

$$d = 3{,}226{,}366{,}849$$

Verify:

$$49 \times 3{,}226{,}366{,}849 \mod 10{,}539{,}465{,}040 = 1$$

$d = 3{,}226{,}366{,}849$, $n = 10{,}539{,}750{,}919$.

---

2. Read Huffman code algorithm from the text or from the Internet. Recall that the algorithm is to code a word in bits (i.e., a binary tree is constructed). Now, we use digits instead of bits, can you design a Huffman code algorithm?

Standard Huffman uses base $D = 2$ (binary). We generalize to any base D. Each internal node has D children, with edges labeled $0, 1, 2, \ldots, D-1$.

For N symbols, we need:

$$(N-1) \equiv 0 \pmod{D-1}$$

If not, add k dummy symbols with frequency 0, where:

$$k = (D-1) - \big((N-1) \bmod (D-1)\big)$$

Count frequencies for all symbols. Pad with zero-frequency dummies if needed. Build a min-heap keyed by frequency. Repeat until 1 node remains: extract the D lowest-frequency nodes, create a parent with $freq = \sum_{i=1}^{D} freq_i$, and push the parent back onto the heap. Assign codes by tracing root to leaf, labeling each branch $0, 1, \ldots, D-1$.

Example with symbols $\{A{:}40,\ B{:}20,\ C{:}15,\ D{:}10,\ E{:}8,\ F{:}5,\ G{:}2\}$, total $= 100$:

Symbol	p	Base-2 code	Base-3 code
A	0.40	0	1
B	0.20	111	0
C	0.15	101	21
D	0.10	100	20
E	0.08	1101	222
F	0.05	11001	221
G	0.02	11000	220

Average codeword length (base D):

$$\bar{L} = \sum_{i} p_i \cdot \ell_i$$

$$\bar{L}_2 = 0.40(1) + 0.20(3) + 0.15(3) + 0.10(3) + 0.08(4) + 0.05(5) + 0.02(5) = 2.42 \text{ bits}$$

$$\bar{L}_3 = 0.40(1) + 0.20(1) + 0.15(2) + 0.10(2) + 0.08(3) + 0.05(3) + 0.02(3) = 1.55 \text{ trits}$$

Entropy lower bound (base D):

$$H_D = -\sum_{i} p_i \log_D p_i$$

$$H_2 = 2.356 \text{ bits}, \quad H_3 = 1.487 \text{ trits}$$

Both average lengths are close to their respective entropy bounds, confirming the algorithm is optimal.

---

3. Try to read from the Internet and summarize (in half page) on current techniques in finding whether a cryptographic protocol has a flaw or not.

The even a well-designed protocol can have subtle flaws. The Needham-Schroeder public-key protocol had a flaw nobody found for 17 years.

The main approach is formal/symbolic verification. The protocol is modeled as message-passing rules under the Dolev-Yao threat model, where the attacker controls the entire network and can intercept, replay, or modify any message M. The security property to verify (e.g. secrecy) is:

$$\nexists \text{ attacker trace } T \text{ such that attacker learns secret } s$$

The top tools are ProVerif (Horn-clause resolution, fully automatic) and Tamarin Prover (multiset rewriting with temporal logic, handles Diffie-Hellman and mutable state).

Even if the protocol design is provably correct, the code can still be broken (e.g. Heartbleed in OpenSSL). Tools extract a formal model directly from C/Java source via static analysis or symbolic execution, then feed it into ProVerif or CryptoVerif.

When the state space is too large for exhaustive search, ML-based frameworks learn dynamic search strategies to guide verification, handling protocols that purely algorithmic tools get stuck on.

In computational/game-based proofs, security is defined probabilistically: no polynomial-time adversary can break the protocol with non-negligible probability ϵ:

$$\Pr[\text{attacker wins}] \leq \epsilon(n)$$

where ϵ is negligible in the security parameter n. CryptoVerif automates some of these reductionist proofs. TLS 1.3 and the 5G authentication protocol have both been formally verified this way using Tamarin.